

SkillMatch AI

Work & data flow specification

Student and employer portals, screen-by-screen workflow, API contracts, and state machines. Formalized from the engineering draft for implementation and thesis documentation.

Field	Detail
Document	Work & Data Flow Specification v1.0
Scope	Student portal, employer portal, notifications, compliance
Companion document	SkillMatch AI Technical Roadmap & Architecture (v1.0)

1. Executive summary & platform topology

SkillMatch AI eliminates recruitment friction between students and employers by turning unstructured resumes into structured, matchable profiles and ranking opportunities against them with a hybrid scoring formula:

```
final_match_score = 0.60 × taxonomy_skill_overlap + 0.40 ×  
cosine_similarity(resume_vector, job_vector)
```

Core ecosystem value pillars

- **Student experience** — frictionless resume upload, automatic skill extraction, human-in-the-loop verification, hybrid-scored discovery feed, live status tracking.
- **Employer experience** — rapid job publishing, automatic skill tagging, real-time AI-ranked applicant review, explainable match breakdowns, one-tap status progression.
- **Data privacy & compliance** — full alignment with the Philippine Data Privacy Act (RA 10173): machine-readable data export and permanent account/file erasure on request.

2. Student portal — end-to-end flow



Figure 1 — student journey, sign-up through status tracking

2.1 Authentication & session initialization

Field	Detail
Screen	LoginScreen.tsx / RegisterScreen.tsx
Workflow	Student enters name, email, password, and role = student. On success the JWT is persisted and attached as the default request header for all subsequent calls.
Endpoint	POST /api/v1/auth/register · POST /api/v1/auth/login
Response	JWT access_token + user object, stored in Expo SecureStore

2.2 Resume ingestion, extraction & embedding

Field	Detail
Screen	ResumeUploadScreen.tsx
Workflow	Student picks a PDF/DOCX (<5MB), taps upload; the backend validates, extracts, and structures the resume while the app shows a parsing state.
Endpoint	POST /api/v1/resumes/upload (multipart/form-data)

Backend transformation pipeline:

- Magic-byte validation (puremagic) rejects renamed/disguised file types before anything else runs.
- File saved to an isolated, sanitized path: storage/resumes/{user_id}/{uuid}.pdf.
- Text extraction via pdfplumber (PDF) / python-docx (DOCX).
- LLM field extraction (OpenAI/Claude, with a heuristic fallback) returns structured JSON: skills, education, experience, certifications, summary.
- Taxonomy normalization maps free-text skills to the canonical dictionary (React.js -> React, py -> Python).
- A 384-dimension dense embedding is generated from the synthesized resume text.

2.3 Human-in-the-loop verification

Field	Detail
Screen	ResumeReviewScreen.tsx
Workflow	Extracted skills render as dismissible chips; the student removes inaccurate tags, adds missing ones, then activates the profile.
Endpoint	PUT /api/v1/resumes/{resume_id}/parsed-data
Backend action	Re-normalizes skills, regenerates the embedding, updates ResumeSkill records, sets status = active

2.4 Recommendation discovery

Field	Detail
Screen	RecommendationFeedScreen.tsx (tab: Opportunities)
Workflow	Feed merges internal employer postings with external synced jobs (JSearch/RapidAPI). Each card shows a match badge, company, job type, location, and an explainability breakdown (matched skills in green, missing skills in amber).
Endpoint	GET /api/v1/matches/recommendations

2.5 Application submission & status tracking

Field	Detail
Screen	RecommendationFeedScreen.tsx (action) → ApplicationTrackerScreen.tsx (view)
Workflow	Tapping Apply Now opens an optional cover-note modal; on confirm the application is created and tracked through a 5-step visual stepper.
Endpoint	POST /api/v1/applications/ · GET /api/v1/applications/my-applications
5-step states	Submitted → Under review → Shortlisted → Interview scheduled → Accepted (or Rejected/Withdrawn)

2.6 In-app status notifications

Field	Detail
Screen	NotificationsScreen.tsx
Workflow	When an employer updates a candidate's status, the backend dispatches a structured in-app notification (type = application_status_update) to the student.
Endpoint	GET /api/v1/notifications/ · GET /api/v1/notifications/unread-count

3. Employer portal — end-to-end flow



Figure 2 — employer journey, registration through status updates

3.1 Employer registration

Field	Detail
Screen	RegisterScreen.tsx (role: employer)
Workflow	Employer signs up with company name, work email, and password. The backend atomically creates the User and a linked EmployerProfile.
Endpoint	POST /api/v1/auth/register (role: employer)

3.2 Job posting creation & vector indexing

Field	Detail
Screen	EmployerPostingsScreen.tsx (modal: Create new job posting)
Workflow	Recruiter fills title, job type, location, remote/on-site toggle, required/preferred skills, and description, then publishes.
Endpoint	POST /api/v1/jobs/
Backend action	Skills normalized via ontology; 384-dim job embedding generated; posting stored with status = active

3.3 Active postings dashboard

Field	Detail
Screen	EmployerPostingsScreen.tsx
Workflow	Recruiter sees all active postings with applicant-count badges and drills into the ranked candidate pipeline for any posting.
Endpoint	GET /api/v1/jobs/my-postings

3.4 AI-ranked applicant review

Field	Detail
Screen	EmployerApplicantsScreen.tsx
Workflow	Applicants are pre-sorted descending by hybrid match score, each with matched/missing skill badges, notes, status, and a resume download link.
Endpoint	GET /api/v1/applications/posting/{posting_id}/applicants

3.5 Recruitment pipeline progression

Field	Detail
Screen	EmployerApplicantsScreen.tsx (action buttons)
Workflow	One-tap status buttons — Shortlist, Schedule interview, Accept, Reject — update the application and immediately notify the student.
Endpoint	PATCH /api/v1/applications/{application_id}/status

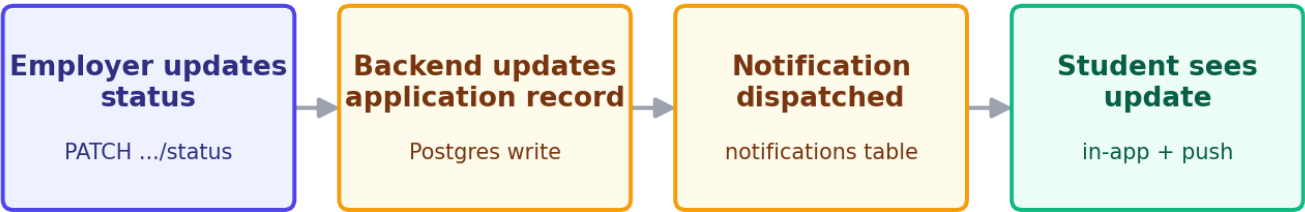


Figure 3 — how an employer action reaches the student (cross-portal data flow)

4. Master API & data flow matrix

All routes, required roles, primary inputs, and side effects, in one reference table.

Module	Route	Auth / role	Input	Output / side effect
Auth	POST /auth/register	Public	email, password, full_name, role	Creates User (+ EmployerProfile if employer); returns JWT
Auth	POST /auth/login	Public	email, password	Returns JWT + user data
Auth	GET /auth/me/export	Student / Employer	none	RA 10173 personal-data export (JSON)
Auth	DELETE /auth/me	Student / Employer	none	RA 10173 cascading delete + disk purge
Resumes	POST /resumes/upload	Student	multipart PDF/DOCX	Extracts text, LLM JSON, embedding
Resumes	PUT /resumes/{id}/parsed-data	Student	parsed_data JSON	Re-normalizes skills & embedding
Resumes	DELETE /resumes/{id}	Student	none	Deletes record and file on disk
Jobs	POST /jobs/	Employer	title, job_type, skills, desc	Creates posting, builds job embedding
Jobs	GET /jobs/my-postings	Employer	none	Active postings + applicant counts
Matches	GET /matches/recommendations	Student	none	Hybrid-scored internal + external feed
Applications	POST /applications/	Student	job_posting_id, notes	Creates application (submitted)
Applications	GET /applications/my-applications	Student	none	Student's applications, 5-step status
Applications	GET /applications/posting/{id}/applicants	Employer	none	AI-ranked applicant pool
Applications	PATCH /applications/{id}/status	Employer	status enum	Transitions status, notifies student
Notifications	GET /notifications/	Authenticated	none	User's notification feed
Evaluation	POST /evaluation/run-benchmark	Coordinator/Admin	none	Precision@K, NDCG@K, MRR benchmark

5. State machines & status enums

5.1 Application status lifecycle

A strict, mostly-linear progression with three terminal states. Rejection or withdrawal can occur from any pre-acceptance stage, not only from Shortlisted.

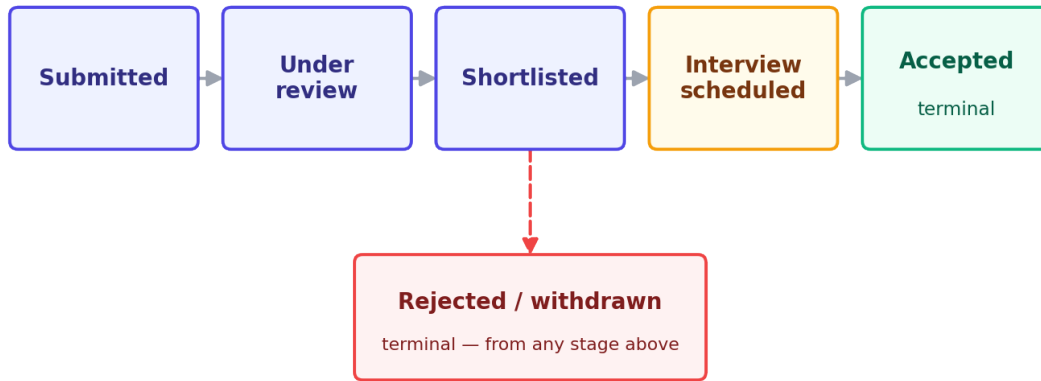


Figure 4 — application status state machine

5.2 Resume parsing status lifecycle

A resume is not eligible for matching until it reaches active — i.e., until the student has reviewed the AI-extracted fields.

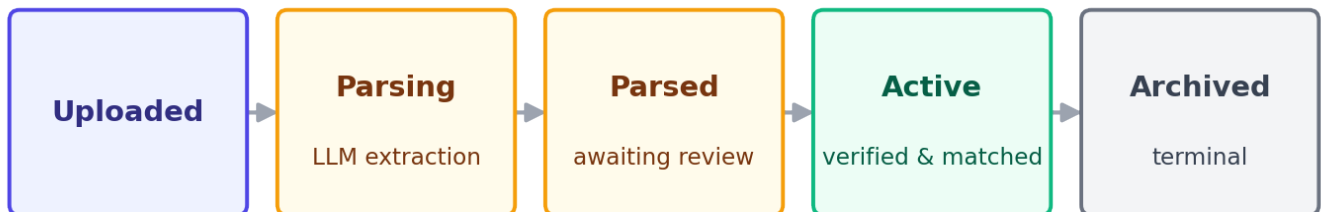


Figure 5 — resume parsing status lifecycle

5.3 Status enum reference

Status	Meaning
<code>submitted</code>	Initial state on application creation.
<code>under_review</code>	Recruiter has opened and viewed the candidate profile.
<code>shortlisted</code>	Marked as high-potential for the interview pool.
<code>interview_scheduled</code>	Candidate invited for a technical/behavioral interview.

accepted	Terminal — offer extended and accepted.
rejected	Terminal — application declined.
withdrawn	Terminal — student voluntarily withdrew.

6. Data privacy & compliance notes

Two endpoints exist specifically to satisfy the Philippine Data Privacy Act of 2012 (RA 10173) and should be treated as first-class, not optional, features:

- **GET /auth/me/export** — returns a complete, machine-readable export of the user's personal data on request (a core data-subject right under RA 10173).
- **DELETE /auth/me** — permanently and cascadingly deletes the account, its database records, and the physical resume file(s) on disk. This must actually reach disk storage, not just soft-delete the database row.

Both endpoints require an authenticated Bearer JWT and act only on the requesting user's own account — never on a target user ID supplied in the request body or query string.

This document formalizes the work & data flow draft into a stable reference for the mobile engineering team. Visual UI mockups for each screen are provided separately as inline references. Pair this with the SkillMatch AI Technical Roadmap & Architecture document for infrastructure, evaluation methodology, and delivery phasing.